

EA3_PROGRAMACIÓN_FUNCIONAL

PROGRAMACIÓN II: ORIENTADA A OBJETOS AVANZADA PREICA2602B010093

BORIS ALBERTO SALLEG

EMILSE CAVADIA TORDECILLA

IDENTIFICACION: CC 1033369434

emilse.cavadia@est.iudigital.edu.co

CRISTIAN CAMILO DEOSSA BOLIVAR

IDENTIFICACION: CC 1047994439

cristian.deossa@est.iudigital.edu.co

JOSE WILBER MURILLO MURILLO

IDENTIFICACION: CC 1010029229

wilber.murillo@est.iudigital.edu.co

KAROL LICETH PEREZ CASTAÑEDA

IDENTIFICACION: CC 101153607

karol.perez@est.iudigital.edu.co

NICOLE VALENTINA QUEVEDO FRANCO

IDENTIFICACION: CC 1025537804

nicole.quevedo@est.iudigital.edu.co

GRUPO 7

INSTITUCION UNIVERSITARIA DIGITAL DE ANTIOQUIA

26/09/2026

1. Introducción

El proyecto TecnoMovil Data busca procesar datos simulados de un sistema de transporte urbano utilizando programación funcional en Java. Para ello, se aplican conceptos como funciones, inmutabilidad, lambdas y Streams, permitiendo analizar la afluencia por estación, horas pico, rutas, patrones de viaje y posibles sobrecargas. Además, el proyecto se desarrolla de manera colaborativa mediante GitHub.

2. Objetivos

2.1. Objetivo general

Aplicar conceptos de programación funcional en Java para procesar de forma declarativa datos simulados de transporte urbano.

2.2 Objetivos específicos

- Representar registros con atributos inmutables.
- Aplicar funciones puras, lambdas y Streams.
- Calcular afluencia y horas pico.
- Analizar rutas y detectar rutas críticas.
- Obtener patrones de viaje y tiempo promedio.
- Documentar la ejecución y la colaboración en GitHub.

3. Descripción general del sistema

El sistema actual usa un enfoque imperativo con estructuras mutables y código acoplado. Esto dificulta procesar grandes volúmenes, detectar patrones y generar estadísticas.

4. Solución propuesta

Se crea un módulo Java sin frameworks externos. Los registros tienen usuario, ruta, estación, acción y fecha/hora. Los análisis se realizan con Streams y lambdas sin modificar la colección de entrada.

5. Conceptos funcionales

5.1. Funciones puras

Los métodos reciben datos y producen resultados sin modificar la lista original.

5.2. Inmutabilidad

Los atributos son final y las colecciones de resultados se protegen.

5.3. Lambdas

Se usan expresiones lambda para filtros y transformaciones.

5.4. Streams

Se usan para filtrar, agrupar, ordenar y contar.

5.5. Funciones de orden superior

contarPor recibe Predicate y Function como parametros.

6. Desarrollo

1. Afluencia por estación.
2. Horas pico.
3. Rutas más utilizadas.
4. Patrones por usuario.

5. Tiempo promedio entre estaciones.
6. Rutas críticas con umbral de cinco entradas.

7. Resultados

Análisis	Resultado
Afluencia	Central 4; Universidad 3; Centro 3; Industrial 2; Aeropuerto 2; Sur 1; Norte 1
Rutas	R1=7; R2=5; R3=4
Umbral	Mayor que 5 entradas
Ruta crítica	R1
Registros	24 simulados

Los datos son simulados. Las horas pico y el tiempo promedio se calculan al ejecutar el programa.

8. Conclusión

La programación funcional permitió organizar el procesamiento del caso de manera clara. Streams y lambdas facilitan los cálculos y la inmutabilidad ayuda a evitar modificaciones accidentales. GitHub permite organizar el trabajo colaborativo mediante ramas, commits, push, pull y merge.

9. Enlaces del repositorio GitHub y video

- **Enlace repositorio GitHub:**

<https://github.com/Karol-perez-castaneda/Evidencia3-Programacion-Funcional>

- **Enlace video:**

https://drive.google.com/file/d/1fLQ-0-fDPUz3rXGyNisAu47lpChNWhur/view?usp=drive_id